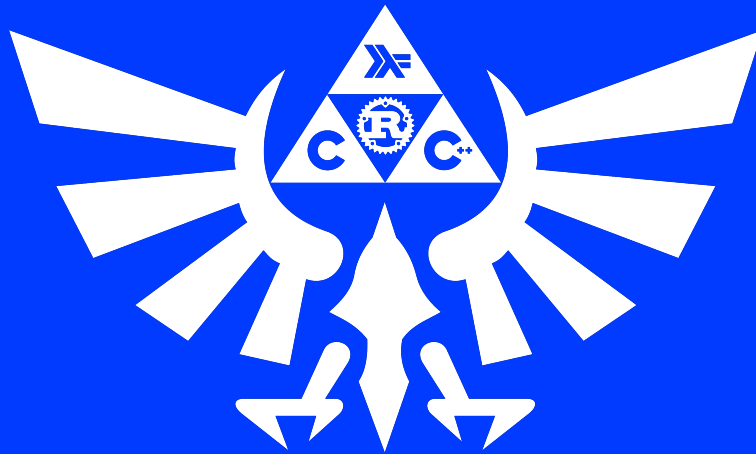


{EPITECH}

BISTROMATIC

< A SIMPLE VIRTUAL MACHINE FOR CASH
REGISTER />



BISTROMATIC

Preliminaries



binary name: bistroMatic

language: C++

compilation: via Makefile, including re, clean and fclean rules

We've been contacted by Epitech's Student Union to design their new cash register system. They gain so much money from the drink vending machine they need a new software to keep count of their cash ! The old one can't do much, so we need you to create a new one.

The physical machine already exists, we only need the software. To make it compatible with any machine, we will interact with it using only simple instructions. So simple it could be considered as a simplified assembly language ! Your register must be an abstract virtual machine that can run on anything (just like **DOOM**).

The goal of this rush is to create a simple virtual machine that is able to interpret commands written in a simplified assembler language. To be precise, it consists in a stack machine that is able to calculate simple arithmetical expressions. These very arithmetical expressions are given to the machine in the form of instructions.

Let's create a calculator.

Specifications

Assembler Language

Here's an example of an assembler program that your machine should be able to execute, that is worth more than all the explanations of the world :

```
; -----  
; - example.bde -  
; -----  
push int32 33  
push int32 42  
add  
display ; probably 75  
push float32 44.55  
mul  
display ; something like 3341.25  
push float64 42.5  
push int32 42  
pop  
display ; logically 42.5  
assert float64 42.5  
exit
```

Like every assembler language, a bistroMatic program is composed of a sequence of instructions, but with only one instruction per line ; a notable difference between other assembler languages.

- ✓ **comments** : They begin with a semicolon ';' and end at the end of a line. A comment may be indifferent at the beginning of a line or after an instruction.
- ✓ **push v** : Stack the v value at the top. The v value will naturally take one of the following forms :
 - int8 n
 - int16 n
 - int32 n
 - int64 n
 - float32 z
 - float64 zfor example : **int16 n** - create an signed 16-bit integer with the value n.
- ✓ **pop** : Unstacks the value at the top of the stack. If the stack is empty the program's execution must stop error.
- ✓ **clear** : Clears the stack, making it empty.
- ✓ **swap** : Reverses the order of (swaps) the top two values on the stack.
- ✓ **display** : Display the value on the top of the stack followed by a newline.
- ✓ **assert v** : Verify that the value at the top of the stack is equal to the one passed as parameter in this instruction. If it's not the case, the program execution must stop error. The v value, of course, has the same form as those passed as parameter during the **push** instruction.
- ✓ **add** : Unstack the first two values in the stack, add them, and then stack the result. If the number of values in the stack is strictly less than 2, the program execution must stop error.
- ✓ **sub** : Unstack the first two values in the stack, subtract them, and then stack the result. If the number

of values in the stack is strictly less than 2, the program execution must stop error.

- ✓ **mul** : Unstack the first two values in the stack, multiply them, and then stack the result. If the number of the values in the stack is strictly less than 2, the program execution must stop error.
- ✓ **div** : Unstack the first two values in the stack, divide them, and then stack the result. If the number of the values in the stack is strictly less than 2, the program execution must stop error. Also, if the divisor is equal to 0, the program execution must also stop error.
- ✓ **mod** : Unstack the first two values in the stack, calculate their modulo, and then stack the result. If the number of the values in the stack is strictly less than 2, the program execution must stop error. Also, if the divisor is equal to 0, the program execution must also stop error.
- ✓ **load n** : Take the value from the register n and move it at the top of the stack. if the register n does not contain a value, the program execution must stop error.
- ✓ **store n** : Unstack the first value and store it to the register n .
- ✓ **exit** : Quit the program execution.



In case you run into non-commutative operations, compute **v2 op v1** (in infix notation), where **v1** is on top of the stack and **v2** under it.

If ever a calculation implies two different operands types, the return value will be the more precise type of operand. Please note that if your machine has extensibility problems, the precision question will not be insignificant. We will look at this in further detail later on in this document.

Our machine has the capacity to store 16 values of any form into registers from 0 to 15. Think of it as variables where we can store values, then re-read the [load](#) and [store](#) instructions.

Grammar

The bistroMatic's assembler language is generated from the following simple grammar (# corresponds to the end of the input not to the # character):

```
S := [INSTR SEP]* #

INSTR := push VALUE
      | pop
      | display
      | clear
      | swap
      | assert VALUE
      | add
      | sub
      | mul
      | div
      | mod
      | load N
      | store N
      | exit

VALUE := int8 N
      | int16 N
      | int32 N
      | int64 N
      | float32 Z
      | float64 Z

N := [-]?[0..9]+

Z := [-]?[0..9]+[.]?[0..9]*

SEP := '\n'
```

Execution

Your machine should be able to execute instructions from the standard input.

```
Terminal
$> ./bistroMatic --help
usage: bistroMatic [options]
options:
-h, --help Show this help
-b, --bonus Add any flag you want for bonuses!
```

Let's check out a few execution examples:

```
Terminal
$> ./bistroMatic < example.bde
75
3341.2500000
42.5000000000000000
```

```
Terminal
$> ./bistroMatic
push int32 2
push int32 3
add
assert int32 5
display
5
CTRL+D
```

```
Terminal
$> ./bistroMatic | cat -e
pop
error: pop on empty stack.
$> echo $?
84
```



The error message is given as an example, feel free to use your own.

Errors

In the following case(s), your bistroMatic must raise an error and correctly stop the program execution:

- ✓ the assembler program has one, or several, lexical or syntactical errors,
- ✓ an instruction is unknown,
- ✓ pop instruction on an empty stack,
- ✓ load instruction on an empty register
- ✓ division/modulo by 0,
- ✓ an assert instruction is not verified,
- ✓ the stack strictly has less than two values during the execution of an arithmetical instruction.

In case of error, your program should return 84.

You could use exceptions to handle errors in you bistroMatic. Search `try`, `catch`, `throw` and `std::exception`. Your exception class(es) should inherit from `std::exception`.



Your program must not stop on an uncaught exception ! It will be considered as a crash.

Technical considerations

In order to help you in the development of your project, you're going to have to comply with the following instructions. Think about why each instruction is given to you. None of them are there by chance or to annoy you.

IOperand Interface

All of your operand classes must IMPERATIVELY implement the IOperand interface below:

```
class IOperand
{
public:
    virtual std::string toString() const = 0; // string that represents the instance

    virtual OperandType getType() const = 0; // returns the type of instance

    virtual IOperand* operator+(const IOperand &rhs) const = 0; // sum
    virtual IOperand* operator-(const IOperand &rhs) const = 0; // difference
    virtual IOperand* operator*(const IOperand &rhs) const = 0; // product
    virtual IOperand* operator/(const IOperand &rhs) const = 0; // quotient
    virtual IOperand* operator%(const IOperand &rhs) const = 0; // modulo

    virtual ~IOperand() = default;
};
```

`Int8Operand` class, `Int16Operand` class, `Int32Operand` class, `Int64Operand` class, `Float32Operand` class and `Float64Operand` class must implement the IOperand interface.

You are free to modify the provided interface, but any modification must be justified during the defense.



It is FORBIDDEN to manipulate pointers or references for each one of these six classes. You are ONLY allowed to manipulate pointers or references to `IOperand`.

IOperand Factory

You must write a class named `OperandFactory` with a **static member function** that will enable you to create new operands in a general manner. This member function should be prototyped as follows :

```
static IOperand* createOperand(OperandType type, const std::string& value);
```


The `OperandType` is an enum that can take the following values: `Int8`, `Int16`, `Int32`, `Int64`, `Float32` and `Float64`.

Depending on the enum's value passed as parameter, the **createOperand** member function will create a new `IOperand` by calling the following **private** static member functions :

```
static IOperand* createInt8(const std::string& value);
static IOperand* createInt16(const std::string& value);
static IOperand* createInt32(const std::string& value);
static IOperand* createInt64(const std::string& value);
static IOperand* createFloat32(const std::string& value);
static IOperand* createFloat64(const std::string& value);
```

Precision

When an operation takes place between two of the same real types, no problem. But what about if the real types are different ? The usual method is to order the types among themselves in precision order. In our machine's case, this will give the following order :

```
Int8 < Int16 < Int32 < Int64 < Float32 < Float64
```

We can use cleverly the `OperandType` to keep track of this order in our machine. During an operation involving two operands of different types, the type of the result of the operation can be determined. In this project, we will consider that the returned type is always the more precise of the two.

`float32` and `float64` values are written using fixed-point notation. The fixed number of digits to display after the decimal point is `float32(7)`, `float64(15)`. Examples :

```
- float32: 2.0/3.0 = 0.6666667
- float64: 2.0/3.0 = 0.6666666666666667
- float32, float64: 50.0/2000.0 = 0.0250000000000000
```



Do you remember `std::stringstream` and `ioanip` ?

The stack

Given that the bistroMatic is a stack machine, it possesses a container that acts like a stack. The choice will seem obvious now, but perhaps you will discover a subtlety that will make you ask yourself the question again later on.



Your container must ONLY contain `IOperand` abstract type pointers. It is **FORBIDDEN** to store real type operands in your container.

Ressource usage

Every ressources used by our machine must be freed even when error occurs that includes memory allocations.

Restrictions

You are generally free to implement what you want. Also, you are allowed to use any C++ standard supported by the last compiler version of the given dump. However, there are a few restrictions:

- ✓ all external libraries are explicitly forbidden except for bonus (a SFML GUI for example ?),
- ✓ you can use the `STL` as much as you like,
- ✓ the only authorized `libc` functions are those that encapsule the call systems and that have no equivalent in C++,
- ✓ "split", "strtowordtab" and the rest of them must NOT be used for parsing code (even the assembler!), use streams !
- ✓ any value that is passed by copy rather than by reference or pointer must be justified,
- ✓ any value that is not `const` and passed as parameter must also be justified,
- ✓ any member function or method that does not modify the standard instance, and that is not `const` must be justified,
- ✓ any conditional statements with more than 3 branches (if ... else if .. else) must be justified,
- ✓ there is no norm in C++. However, any code that we find to be illegible or too messy will be penalized. Be serious !

Bonus

You can do any bonus you want for this project, but don't forget : a good architecture without bonus is better than a bad architecture with bonus.

Here is some ideas for you bonus :

- ✓ support of the BigInt type, that can handle infinitely big integers
- ✓ support of the BigFloat type, that can handle infinitely big floating point numbers
- ✓ a SFML GUI for the cash register

v 4.3

{EPITECH}